

Citation Validation as Set Operations, Worked Out by Hand

Why a hallucinated citation is just one element of a set difference — and why a valid report is exactly the case where that difference is empty.

What this document does. It takes the final **REPORT** stage of the Module 2.3 research engine — the deterministic, no-LLM step that assembles a **ResearchReport** and runs the *citation round-trip check* — and recasts that check as plain set algebra. We fix one tiny example: a report drew from five fetched sources but its themes cite the ids $\{2, 3, 3, 6\}$. From those two sets alone we compute, by hand, which citations are real, which are hallucinated, which sources went unused, and the two quality metrics (precision and coverage). Every number below is reproduced by the small Python program in Appendix A — and that program is, line for line, the same set-difference test the module’s Pydantic `field_validator` performs to *reject* a report.

Where this sits. This is **Topic 1 of Module 2.3** (Research Summarization Engine) — the formal core of the **REPORT** stage and its “citation indices must round-trip — no orphan refs” constraint. Its companions: Module 1.0 (Pydantic schemas as typed contracts, where the validator lives) and Module 2.5 (RAG-as-a-tool, where the source set V stops being a black box).

Concepts covered, in order: the valid set V and the cited set C · a multiset of citations collapsing to a set · Grounded $= C \cap V$ · Orphans $= C \setminus V$ (hallucinated) · Uncited $= V \setminus C$ (wasted fetches) · citation precision $|C \cap V|/|C|$ · source coverage $|C \cap V|/|V|$ · the round-trip test $C \setminus V = \emptyset \Leftrightarrow \text{precision} = 1$ · how the validator turns that one line into an accept/reject gate.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: two sets, small enough to count by hand

The research engine ends with a stage that takes no model call at all. By the time **REPORT** runs, two things already exist: a list of **sources** the **FETCH** stage actually retrieved (each with an integer **id**), and a list of **themes** the **REDUCE** stage wrote, each carrying a **cited_sources** list of the ids it referenced inline as `[N]` tokens. The job of **REPORT** is to confirm that every `[N]` a theme cites actually points at a real source — the “citation round-trip”. That confirmation is a set operation.

Give the two collections names. Let V (*valid*) be the set of ids that genuinely exist — the fetched sources. Let C (*cited*) be the set of ids the report’s themes reference. Then three regions fall straight out, and two ratios summarise the report’s honesty.

Quantity	Symbol	Value here
Valid source ids (fetched, real)	V	$\{1, 2, 3, 4, 5\}$
Citations the report emits (a multiset)	—	$\langle 2, 3, 3, 6 \rangle$
Cited ids, as a set	C	$\{2, 3, 6\}$
Grounded (cited <i>and</i> real)	$C \cap V$	$\{2, 3\}$
Orphans (cited but nonexistent)	$C \setminus V$	$\{6\}$
Uncited (real but unused)	$V \setminus C$	$\{1, 4, 5\}$

Notation. A theme may cite the same id twice — in our example the token [3] appears twice across the prose, so the raw citations form a *multiset* $\langle 2, 3, 3, 6 \rangle$. For *validity* we only care about *which* ids appear, so we collapse to the set $C = \{2, 3, 6\}$. $|X|$ is the number of distinct elements of X . The symbol $\emptyset$ is the empty set.

Intuition

A citation is a promise: “this claim is backed by source N .” The fetched sources V are the only ids that promise can be kept against. So checking a report is not about reading prose — it is about asking, of the set of ids it pointed at, *which ones do not exist*. That question is literally $C \setminus V$. If the answer is empty, every promise can be kept. If it contains anything, the report cited a source that was never fetched — a hallucinated citation.

2 Step 1 — collapse the multiset, then partition

The report’s prose cited ids $\langle 2, 3, 3, 6 \rangle$. The first move is to discard multiplicity: a source is either referenced or not, and citing it twice does not make it more real. So $C = \{2, 3, 6\}$, with $|C| = 3$ distinct ids even though four [N] tokens were written.

Now lay C and V side by side. Every id in the picture lands in exactly one of three buckets, because set difference and intersection partition the union:

$$(C \setminus V) \sqcup (C \cap V) \sqcup (V \setminus C) = C \cup V, \quad (1)$$

and the three pieces are pairwise disjoint. Walk our ids through it:

id	in C ?	in V ?	bucket
1	no	yes	$V \setminus C$ (uncited)
2	yes	yes	$C \cap V$ (grounded)
3	yes	yes	$C \cap V$ (grounded)
4	no	yes	$V \setminus C$ (uncited)
5	no	yes	$V \setminus C$ (uncited)
6	yes	no	$C \setminus V$ (orphan)

So Grounded = $\{2, 3\}$, Orphans = $\{6\}$, Uncited = $\{1, 4, 5\}$. ✓ Sizes: $|C \cap V| = 2$, $|C \setminus V| = 1$, $|V \setminus C| = 3$, and $2 + 1 + 3 = 6 = |C \cup V|$ — the partition checks out.

Mini-example — this operation in isolation

Orphan detection in isolation is one subtraction. We are not comparing strings or re-reading the sources; we ask set-membership six times. Id 6 is the only cited id that fails “is it in V ?”. That is the entire hallucination check: $C \setminus V = \{6\} \neq \emptyset$. Everything else in this document is just naming and counting the leftovers.

3 Step 2 — the two metrics

Two ratios turn the partition into a score. **Citation precision** asks: of the ids the report cited, what fraction are real?

$$\text{precision} = \frac{|C \cap V|}{|C|} = \frac{2}{3} \approx 0.6667.$$

Source coverage asks the dual question: of the ids we fetched, what fraction actually got used?

$$\text{coverage} = \frac{|C \cap V|}{|V|} = \frac{2}{5} = 0.4000.$$

Both share the numerator $|C \cap V| = 2$ — the grounded ids are the only ones that count toward either score. They differ only in the denominator: precision divides by what was *cited* ($|C| = 3$), coverage by what was *available* ($|V| = 5$). Sample check: precision = $2/3 = 0.6667$; the missing $1/3$ is exactly the orphan id 6 pulling the numerator below the denominator. ✓

Intuition

Precision and coverage fail in opposite directions, and only one of them is a correctness bug. Low *precision* means the report invented citations — that is a hallucination, and it must block publication. Low *coverage* means you fetched sources nobody used — wasteful, maybe a planning smell, but the report is still *honest*. The gate below keys on precision, not coverage: you can ship a report that ignored half its sources; you cannot ship one that cited a source that does not exist.

4 Step 3 — the round-trip check as one equivalence

“Citation round-trip” has a one-line definition. A report is *valid* exactly when it has no orphan citations:

$$\text{valid}(C, V) \iff C \setminus V = \emptyset \iff \text{precision} = 1 \quad (2)$$

The two right-hand conditions are the same fact seen twice. If $C \setminus V = \emptyset$ then every cited id is in V , so $C \cap V = C$, so $|C \cap V|/|C| = 1$. Conversely if precision = 1 then $|C \cap V| = |C|$, and since $C \cap V \subseteq C$ a subset of equal size *is* the whole set, forcing $C \setminus V = \emptyset$. Orphans-empty and precision-one are logically equivalent — one is the boolean gate, the other its score.

Apply it to our example: $C \setminus V = \{6\} \neq \emptyset$, so the report is **invalid** and precision = $2/3 < 1$. ✓ To make it valid, the only legal move is to remove the unbacked claim: drop the [6] citation. Then $C = \{2, 3\}$, $C \setminus V = \emptyset$, and precision = $2/2 = 1$. ✓ Coverage rises too, to $2/5$ unchanged in numerator — removing a fake citation never costs you a real one.

Mini-example — this operation in isolation

The fix is a set operation, not a rewrite. “Repair the report” means: compute $C \setminus V$, and for each orphan id, either delete the claim that cites it or re-fetch a source to back it (growing V until the orphan is absorbed). Both routes drive $C \setminus V$ to $\emptyset$. There is no third option that keeps the orphan and stays valid — the equivalence in Eq. (2) leaves no room.

5 Step 4 — the validator is this subtraction

In the module, this check is not prose advice; it is enforced at the type boundary. The **ResearchReport** schema carries a Pydantic `field_validator` on `themes` that runs exactly $C \setminus V$ per theme and

raises if it is non-empty:

```

1 @field_validator("themes")
2 def _citations_resolve(cls, themes, info):
3     """Reject reports with orphan citation indices."""
4     valid_ids = {s.id for s in info.data.get("sources", [])} # this is V
5     for t in themes:
6         unknown = set(t.cited_sources) - valid_ids # this is C_t \ V
7         if unknown:
8             raise ValueError(f"theme '{t.name}' cites unknown sources: {unknown}")
9     return themes

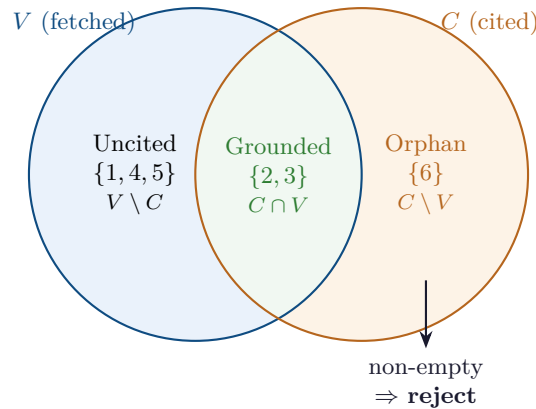
```

Line by line: `valid_ids` is our V , built once from the fetched `sources`. `set(t.cited_sources)` collapses one theme’s citation multiset to its set C_t . The expression `set(t.cited_sources) - valid_ids` is $C_t \setminus V$ — the orphan set for that theme. The `if unknown` test is the predicate $C_t \setminus V \neq \emptyset$ from Equation (2); when it fires, the object refuses to construct. The report-level orphan set is the union over themes, $\bigcup_t (C_t \setminus V)$, and the report is valid iff that union is empty.

Watch out

The validator keys on *precision*, never coverage — and that asymmetry is deliberate. There is no check that $V \setminus C = \emptyset$, because an unused source is not an error: it is fine to fetch five and cite two. If you mistakenly added a “coverage must be 1” rule, every honest report that pruned a weak source would fail to construct. The schema enforces the one invariant that protects the reader (no fake citations) and stays silent on the one that is merely an efficiency hint (unused fetches).

6 The check, drawn



Two overlapping circles. The blue-only crescent is fetched-but-unused (harmless); the green lens is the grounded ids that back real claims; the orange crescent — cited but outside V — is the orphan. A valid report is one whose orange crescent is empty. The picture is Equation (2).

7 Citation cheat-sheet

Valid (fetched) ids	V
Cited ids (multiset collapsed to set)	C
Grounded citations	$C \cap V$
Orphan (hallucinated) citations	$C \setminus V$
Uncited (unused) sources	$V \setminus C$
Citation precision	$\frac{ C \cap V }{ C }$
Source coverage	$\frac{ C \cap V }{ V }$
Round-trip valid	$C \setminus V = \emptyset \Leftrightarrow \text{precision} = 1$
Validator (per theme)	<code>set(cited) - valid_ids</code> must be empty

8 An honest word about these numbers

The sets here ($V = \{1, \dots, 5\}$, citations $\langle 2, 3, 3, 6 \rangle$) are tiny illustrative choices, picked so you can run the membership test in your head — the module's real run fetched 24 unique sources with ids 0..23 and cited a handful per theme. What is *not* illustrative is the structure. Whatever the sizes, a citation is valid iff its id lies in the fetched set, the hallucinated citations are exactly $C \setminus V$, and the report ships iff that difference is empty. Precision and coverage are just the two ways to normalise $|C \cap V|$. Change the ids and the example moves; the set algebra — and the one-line validator that enforces it — does not. That is why this check is the cheap, deterministic, no-LLM final stage: correctness here is a property of two integer sets, nothing more.

Appendix A — Reference implementation

Every number in this document is produced by the program below. It collapses the citation multiset, partitions the ids into grounded / orphan / uncited, computes precision and coverage, and runs the same orphan-set test the module's `field_validator` uses to accept or reject a report.

```

1 # citation_setops.py -- reproduces every number in "Citation Validation as Set Operations".
2
3 V = {1, 2, 3, 4, 5} # valid source ids: the fetched sources
4 cited_multiset = [2, 3, 3, 6] # raw [N] tokens the report's themes emitted
5 C = set(cited_multiset) # collapse multiset -> set {2, 3, 6}
6
7 grounded = C & V # cited AND real -> C cap V
8 orphans = C - V # cited but nonexistent -> C \ V (hallucinated)
9 uncited = V - C # real but unused -> V \ C
10
11 precision = len(grounded) / len(C) # |C cap V| / |C|
12 coverage = len(grounded) / len(V) # |C cap V| / |V|
13
14 print(f"C (set) = {sorted(C)}")
15 print(f"grounded = {sorted(grounded)} |C cap V| = {len(grounded)}")
16 print(f"orphans = {sorted(orphans)}")
17 print(f"uncited = {sorted(uncited)}")
18 print(f"precision = {len(grounded)}/{len(C)} = {precision:.4f}")

```

```
19 print(f"coverage = {len(grounded)}/{len(V)} = {coverage:.4f}")
20 print(f"valid report (orphans empty)? {orphans == set()}")
21
22 # the round-trip gate, exactly as the Pydantic field_validator runs it:
23 def report_is_valid(cited_ids, valid_ids):
24     return set(cited_ids) - set(valid_ids) == set() # C \ V == empty
25
26 print(f"gate(before) = {report_is_valid(cited_multiset, V)}")
27 fixed = [2, 3] # drop the orphan [6]
28 print(f"gate(after dropping 6) = {report_is_valid(fixed, V)}"
29       f" precision = {len(set(fixed) & V)}/{len(set(fixed))}")
30
31 # Expected output:
32 # C (set) = [2, 3, 6]
33 # grounded = [2, 3] |C cap V| = 2
34 # orphans = [6]
35 # uncited = [1, 4, 5]
36 # precision = 2/3 = 0.6667
37 # coverage = 2/5 = 0.4000
38 # valid report (orphans empty)? False
39 # gate(before) = False
40 # gate(after dropping 6) = True precision = 2/2
```

— end of document —